

Scaling techniques applied to the customer churn dataset

WILDA Team 4 · ACS Work Integrated Learning, Data Analytics · Stage 2 Data Preparation · Rohan Sharma Kharel

1. Why the data needs scaling

After encoding, the dataset has 10 features and one target. Eight of the features are 0/1 flags. The other two are much larger. **tenure** runs from 0 to 72 months, and **MonthlyCharges** runs from \$18 to \$119.

The clustering stage uses K-Means, which puts each customer in the cluster with the nearest centre. Unscaled, a \$50 difference in monthly charge counts 50 times as much as the difference between having and not having phone service. The clusters would mostly split customers by price. The Stage 3 neural network also trains more slowly and less reliably when its inputs have very different ranges. Scaling puts tenure and MonthlyCharges on a range close to the 0/1 flags, so no feature counts more just because of its units.

2. Technique chosen: StandardScaler

StandardScaler replaces each value with its distance from the column mean, measured in standard deviations: $z = (x - \text{mean}) / \text{std}$. Each scaled column has a mean of 0 and a standard deviation of 1. We compared it with two other scalers:

- **MinMaxScaler** maps each column to the range 0 to 1 using its minimum and maximum, so the single most extreme customer at each end sets the scale for everyone. StandardScaler uses the mean and standard deviation of every row, and it centres each column on 0, which suits the Stage 3 neural network.
- **RobustScaler** uses the median and the interquartile range, which helps when a few extreme outliers distort the mean and standard deviation. These two columns have no such outliers. Tenure never goes above 72 months and MonthlyCharges never goes above \$119.

3. Columns scaled and not scaled

The pipeline scales only **tenure** and **MonthlyCharges**. The eight encoded columns (gender, SeniorCitizen, Dependents, PhoneService, MultipleLines, InternetService, Contract_One_year and Contract_Two_year) already hold 0 or 1, which is close to the range of the scaled columns. Scaling a 0/1 column would swap its two values for two other numbers. The model would learn nothing new, and the column would be harder to read. The pipeline never scales the target, **Churn**, because it is the value the model predicts rather than an input.

4. Preventing data leakage

The pipeline splits the data first and scales it second. It fits the scaler on the training set only, then uses that fitted scaler to transform the test set.

If we fitted the scaler on all 7,043 rows, the mean and standard deviation it uses would come partly from the test rows. The test set stands in for customers the model has never seen. Once its statistics shape the training data, the test results look better than the model would do on real new customers. Fitting on the training set only also matches how the model will be used. When a new customer joins, the only statistics available are the ones learned from the training data.

5. Training and test split

We split the dataset 80/20, stratified on Churn, with random_state=42 so the split is the same on every run. Only 26.54% of customers churned, and stratifying keeps that rate the same in both sets.

Set	Rows	% of total	Churners	Churn rate
Full dataset	7,043	100.00%	1,869	26.54%
Training set	5,634	79.99%	1,495	26.54%
Test set	1,409	20.01%	374	26.54%

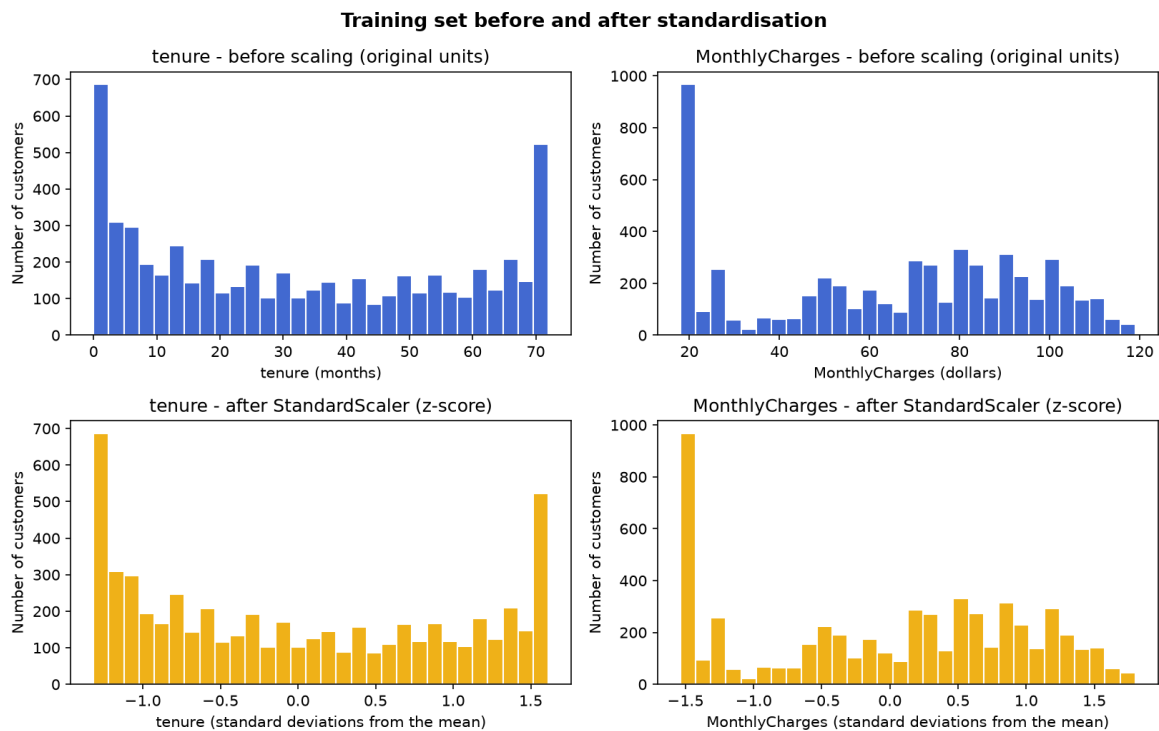
6. Results

Column	Train mean before	Train std before	Train mean after	Train std after	Test mean after	Test std after
tenure (months)	32.3603	24.5639	0.0000	1.0000	0.0022	0.9987
MonthlyCharges (\$)	64.5351	30.2583	0.0000	1.0000	0.0369	0.9712

Both training columns have a mean of 0 and a standard deviation of 1, because the scaler learned its statistics from that set. The test columns come out close to those values but not equal to them. Tenure has mean 0.0022 and standard deviation 0.9987, and MonthlyCharges has mean 0.0369 and standard deviation 0.9712.

This gap is expected, and it is a useful check. If the test columns also came out at exactly 0 and 1, the scaler would have learned from the test rows.

The figure below shows the training set before and after scaling. The bars have the same shape in both rows. Only the axis changes, from months and dollars to standard deviations from the mean.



Training set distributions before (top) and after (bottom) standardisation.

7. Code snippets

These snippets come from `Data_Preparation/data_preparation.py`. The pipeline splits the data before it scales anything:

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=TEST_SIZE, random_state=RANDOM_STATE, stratify=y
)
```

It then calls `fit_transform` on the training rows only. The test rows go through `transform`, which reuses the mean and standard deviation learned from the training rows:

```
scaler = StandardScaler()
X_train[SCALE_COLS] = scaler.fit_transform(X_train[SCALE_COLS]) # fit on TRAIN
X_test[SCALE_COLS] = scaler.transform(X_test[SCALE_COLS])        # apply to TEST
```

Finally, it saves the fitted scaler:

```
joblib.dump(scaler, OUT_DIR / "scaler.pkl")
```

8. Reusing the scaler in later stages

`scaler.pkl` stores the mean and standard deviation learned from the training set. For tenure these are 32.3603 and 24.5639, and for MonthlyCharges 64.5351 and 30.2583.

The clustering stage and the Stage 3 model load this file instead of fitting their own scaler, so every stage scales data the same way. The same file converts a cluster centre back into months and dollars for the client:

```
import joblib

scaler = joblib.load("Data_Preparation/scaler.pkl")
SCALE_COLS = ["tenure", "MonthlyCharges"]

# Scale new data the same way as the training data
new_data[SCALE_COLS] = scaler.transform(new_data[SCALE_COLS])

# Or read a scaled value back in months and dollars
original = scaler.inverse_transform(scaled_values)
```

The repository already contains the scaled sets as `train_set.csv` and `test_set.csv`, so later stages can use them directly. They need the scaler only for new data and for converting results back to months and dollars.

The pipeline writes `preparation_summary.json` on each run, and every row count, churn rate and scaler statistic in this document comes from that file.